

Security Review

Cantina Managed

BERACHAIN: CONTRACTS- METAMORPHO-V1.1

Cantina Managed review by:

Matías Barrios, Security Researcher

Jonatas Martins, Security Researcher

Jonatasmartins

Matiasbn



September 28, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Single-step ownership transfer in <code>MetaFeePartitioner</code> can permanently freeze the fee configuration of every vault	4
3.1.2	A live <code>renounceOwnership</code> in <code>MetaFeePartitioner</code> lets one call permanently disable the fee configuration and the upgrade path	4
3.1.3	<code>MetaFeePartitioner</code> has no way to clear a vault's percentage override and put it back on the default	5
3.1.4	Compromised fee partitioner can block vault operations	5
3.2	Gas Optimization	6
3.2.1	Redundant fee recipient checks in fee distribution	6
3.3	Informational	7
3.3.1	Unused import and incomplete <code>NatSpec</code>	7
3.3.2	Platform fee recipient changes don't apply to pending vault fees	7
3.3.3	Factory cannot replace a compromised fee partitioner	8



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

From Sep 10th to Sep 10th the Cantina team conducted a review of `contracts-metamorpho-v1.1` on commit hash `1419e83b`. The team identified a total of **8** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	1	3
Gas Optimizations	1	1	0
Informational	3	1	2
Total	8	3	5

2.1 Scope

The security review had the following components in scope for `contracts-metamorpho-v1.1` on commit hash `1419e83b`:

```
src
├─ MetaMorphoV1_1.sol
├─ libraries
│   └─ ErrorsLib.sol
```

3 Findings

3.1 Low Risk

3.1.1 Single-step ownership transfer in `MetaFeePartitioner` can permanently freeze the fee configuration of every vault

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Description: `MetaFeePartitioner` inherits `OwnableUpgradeable`, whose `transferOwnership` moves ownership in one transaction with no acceptance step from the incoming address. The owner is the only party that can call `setFeePercentage`, `setDefaultPlatformFeePercentage` and `_authorizeUpgrade`, and every vault holds the partitioner as the immutable `FEE_PARTITIONER` with no setter, so no vault can point elsewhere if that ownership is lost.

A transfer to an address the recipient does not control therefore has no recovery path. The consequence is not limited to configuration: because `_accrueInterest` calls `FEE_PARTITIONER.getShares` on every accruing entry point, an upgrade is the only way to repair a partitioner that starts reverting, and an unreachable owner removes that repair.

Risk: the fee split of every deployed vault becomes permanently unchangeable, and a later fault in the partitioner becomes unfixable, blocking deposits and withdrawals across all vaults with no way to restore them.

Recommendation: Inherit `Ownable2StepUpgradeable` instead of `OwnableUpgradeable`, so a transfer requires the incoming owner to call `acceptOwnership` and a mistyped address leaves the current owner in place.

Berachain: Acknowledged. We have no plans to change ownership at this time, and Berachain governance will retain its ownership status.

Cantina: Acknowledged.

3.1.2 A live `renounceOwnership` in `MetaFeePartitioner` lets one call permanently disable the fee configuration and the upgrade path

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Description: `MetaFeePartitioner` inherits `OwnableUpgradeable` and does not override `renounceOwnership`, so the owner can set the owner to `address(0)` in a single call. The owner is the only party allowed to call `setFeePercentage`, `setDefaultPlatformFeePercentage` and `_authorizeUpgrade`, and the state that follows is terminal: `initialize` reverts `InvalidInitialization` on an already initialized proxy, so ownership cannot be re-established.

Every vault holds the partitioner proxy as the immutable `FEE_PARTITIONER` with no setter, so a vault cannot point elsewhere once that happens. The upgrade path matters beyond configuration: `_accrueInterest` calls `FEE_PARTITIONER.getShares` on every accruing entry point, so replacing the implementation is the only way to repair a partitioner that starts reverting or returning a non-conserving split, and that repair disappears with the owner.

The function serves no purpose here. Renouncing is meaningful for a contract that is intended to become immutable and self-sufficient, while this one is a configuration authority that vaults depend on for their fee split and for their own liveness.

Risk: one call permanently fixes the fee split of every deployed vault at its current values and removes the only means of fixing a later fault in the partitioner, which would leave deposits and withdrawals blocked across all vaults with no recovery.



Recommendation: Override `renounceOwnership` to revert, so ownership can only ever move to another address:

```
function renounceOwnership() public view override onlyOwner {
    revert ErrorsLib.NotSupported();
}
```

Berachain: Acknowledged. Berachain governance will not renounce the ownership.

Cantina: Acknowledged.

3.1.3 `MetaFeePartitioner` has no way to clear a vault's percentage override and put it back on the default

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Description: `setFeePercentage` writes `fee.isSet = true` for a vault and nothing ever writes it back to `false`. `getPlatformPercentage` branches on that flag alone, returning the stored `platformPercentage` when it is set and `defaultPlatformFeePercentage` only when it is not.

Applying a specific percentage per vault is the intended model, so a configured vault holding its own value is not itself a problem. The gap is that the decision is one-way: once a vault has been configured, governance has no way to put it back under `defaultPlatformFeePercentage`.

Risk: a vault that governance wants back on the default platform percentage cannot be returned to it, so the only way to keep it aligned is to rewrite its percentage by hand after every change of the default.

Recommendation: Add an owner-only way to clear a vault's override, for example:

```
function unsetFeePercentage(address vault) external onlyOwner {
    delete _vaultFeePercentages[vault];
    emit EventsLib.FeePercentageUnset(vault);
}
```

Berachain: Acknowledged. Currently unneeded due to business requirements.

Cantina: Acknowledged.

3.1.4 Compromised fee partitioner can block vault operations

Severity: Low Risk.

Context: `MetaMorphoV1_1.sol#L955`.

Description: The vault calls `FEE_PARTITIONER.getShares()` during `_accrueInterest()` whenever `feeShares != 0` and both the platform and vault fee recipients are nonzero. A compromised partitioner can cause this call or the subsequent consistency check to revert, blocking the vault operations that depend on fee accrual:

```
(platformShare, recipientShare) = FEE_PARTITIONER.getShares(address(this),
    ↪ feeShares);
if (platformShare + recipientShare != feeShares) revert
    ↪ ErrorsLib.InconsistentFeePartitioning();
```

The consistency check prevents excessive fee-share minting, but it does not prevent DOS. An attacker who compromises the partitioner's upgrade authority can install a malicious implementation through the owner-authorized upgrade mechanism and trigger any of the following scenarios:



1. **Return invalid fee allocations.** The implementation returns values for which `platformShare + recipientShare != feeShares`, causing `InconsistentFeePartitioning` to revert the transaction. For example, returning `(feeShares, 1)` prevents accrual from completing. Returning values whose sum overflows also causes a revert.
2. **Remove or break `getShares()`**. The attacker upgrades to an implementation that no longer supports `getShares(address,uint256)`. Without a compatible fallback, the call reverts or returns data that the vault cannot decode. An implementation that deliberately reverts produces the same result.
3. **Exhaust the gas forwarded to the call.** The implementation performs unbounded computation, such as an infinite loop, until `getShares()` runs out of gas. The vault does not catch the failed external call, so `_accrueInterest()` also reverts. Exhausting the gas forwarded to the partitioner is sufficient; the attacker does not need to consume all of the caller's remaining gas.

All three scenarios can block `deposit()`, `mint()`, `withdraw()`, `redeem()`, `setFee()`, and `setFeeRecipient()` while the call conditions are met. Users can therefore be prevented from withdrawing through the vault. The vault owner cannot simply disable fees with `setFee(0)`, because this function accrues fees using the current configuration before updating the fee.

This scenario requires compromise of the partitioner's upgrade authority.

Recommendation: Call `getShares()` through a low-level `call` with a fixed gas limit and handle its success flag. If the call reverts, exhausts its allotted gas, returns malformed data, or returns inconsistent allocations, assign all fee shares to the vault fee recipient:

```
platformShare = 0;
recipientShare = feeShares;
```

Limit return-data copying to the expected 64 bytes and validate the response length before decoding. Choose a gas limit that accommodates the intended partitioner implementation while leaving enough gas for the vault to complete the fallback and mint the fee shares. This allows fee accrual to continue when the partitioner fails, with the vault fee recipient receiving the full fee for that accrual.

Berachain: Fixed in [commit 97839df](#).

Cantina: Fix verified.

3.2 Gas Optimization

3.2.1 Redundant fee recipient checks in fee distribution

Severity: Gas Optimization.

Context: [MetaMorphoV1_1.sol#L946](#).

Description: The vault requires a nonzero `feeRecipient` whenever its performance fee is enabled. `setFee()` rejects a positive fee if no recipient is configured, and `setFeeRecipient()` rejects the zero address while the fee is positive.

`_accruedFeeAndAssets()` only computes positive `feeShares` when `fee != 0`. Therefore, `feeShares != 0` also guarantees that `feeRecipient != address(0)`.

This makes the recipient check in `_accrueInterest()` redundant:

```
if (feeShares != 0 && (platformFeeRecipient != address(0) || feeRecipient !=
    → address(0))) {
```

Whenever `feeShares != 0`, the expression inside the parentheses is already true. For the same reason, the following branch cannot be reached:



```
else if (feeRecipient == address(0)) {
    platformShare = feeShares;
}
```

Recommendation: Simplify the outer condition to `if (feeShares != 0)` and remove the unreachable `feeRecipient == address(0)` branch.

Berachain: Fixed in [commit 80b21a7](#).

Cantina: Fix verified.

3.3 Informational

3.3.1 Unused import and incomplete NatSpec

Severity: Informational.

Context: [IMetaMorphoV1_1.sol#L7](#), [MetaMorphoV1_1Factory.sol#L34](#).

Description:

1. **Unused import in `IMetaMorphoV1_1`** . The reviewed version of `src/interfaces/IMetaMorphoV1_1.sol` imports `IMetaFeePartitioner` through `import {IMetaFeePartitioner} from "../IMetaFeePartitioner.sol";`, but does not reference the imported file.
2. **Missing NatSpec for `feePartitioner` in `MetaMorphoV1_1Factory`** . The constructor documentation in `src/MetaMorphoV1_1Factory.sol` describes the `morpho` parameter but omits `feePartitioner` .

Recommendation:

1. Remove the unused `IMetaFeePartitioner` import from `IMetaMorphoV1_1.sol` .
2. Add the following NatSpec entry to the factory constructor.

Berachain: Fixed in [commit 510eaba](#).

Cantina: Fix verified.

3.3.2 Platform fee recipient changes don't apply to pending vault fees

Severity: Informational.

Context: [MetaMorphoV1_1.sol#L944](#).

Description: The vault reads `platformFeeRecipient` directly from `MORPHO.feeRecipient()` whenever `_accrueInterest()` runs. Changing this address in Morpho does not first settle pending fees in the vaults. As a result, the next vault accrual uses the new platform recipient for all pending platform fees, including those earned before the address changed.

The vault's own `setFeeRecipient()` handles this differently: it calls `_accrueInterest()` before updating `feeRecipient` , so the previous recipient receives the fees earned up to that point. Morpho's `setFeeRecipient()` only updates its stored address and emits an event.

This is an informational note about fee allocation during recipient changes. The platform recipient is global, while vaults settle their fees independently. Accruing fees for every outstanding vault within the recipient update would require processing an unbounded number of vaults and is not a practical requirement.



Recommendation: Document that changing `MORPHO.feeRecipient()` also changes the recipient of any platform fees that MetaMorpho vaults have not yet settled, including fees earned before the update.

Berachain: Acknowledged.

Cantina: Acknowledged.

3.3.3 Factory cannot replace a compromised fee partitioner

Severity: Informational.

Context: `MetaMorphoV1_1Factory.sol#L24`.

Description: The factory stores `FEE_PARTITIONER` as an immutable address and passes it to every vault it creates. It has no function to replace this dependency or pause vault creation if the partitioner becomes compromised.

Although the address is fixed, the partitioner's implementation can change. An attacker who compromises the `FEE_PARTITIONER` can replace the partitioner's implementation while keeping the same address. Users can continue creating vaults through the factory, and those vaults will inherit the compromised dependency.

This finding assumes compromise of the partitioner's upgrade authority and concerns the factory's inability to switch to a safe replacement afterward. Each created vault also stores its partitioner address immutably, so changing the factory's configuration would only protect vaults created after the update.

Recommendation: Although contract size is a constraint, the recommendation is to replace the immutable partitioner address with an internal storage variable and add an external function to update it.

Berachain: Acknowledged. Due to size limits, we cannot expand the contract's functions at this time. Additionally, since Berachain governance owns both contracts, a breach in the FeePartitioner would also compromise the Factory. In the event of an exploit, we will deploy a new Factory and filter the affected vaults directly in the UI.

Cantina: Acknowledged.